Computer Networks Lab (CSE/PC/B/S/314): Assignment 2 Report

Flow Control at the Data Link Layer: Stop-and-Wait, Go-Back-N and Selective Repeat ARQ

1. Title

Field	Details
Name	Soumyadip Modak (302510501001)
Class	BCSE - III/ 3 Year / 1 Semester)
Group	A1
Assignment number	2 (CO2)
Course	CSE/PC/B/S/314 Computer Networks Lab
Code demonstration deadline	24-28 August 2026
Report submission deadline	31 August - 4 September 2026
Date of submission	
Language / platform	C, POSIX UDP sockets (Linux)

Problem statement. Implement flow control for the Data Link Layer. A Sender frames a text file, passes every frame through a Channel that can lose or corrupt it, and transmits it to a Receiver. The Receiver checks each frame with the Checksum and CRC modules of Assignment 1, discards bad frames and acknowledges good ones. Three schemes are implemented and compared: Stop-and-Wait, Go-Back-N and Selective Repeat ARQ. The comparison covers the time between sending a frame and receiving its ACK, efficiency without errors, and efficiency for error probabilities from 0.1 to 0.5.

2. Design

2.1 Purpose of the program

The program simulates an unreliable link between two stations over UDP (port 4455). The **Sender (Client_1.c)** reads the input file in 46-byte blocks, builds frames, applies the Channel model and runs one of three ARQ procedures. The **Receiver (Server_1.c)** verifies every frame and replies with a 2-byte ACK according to the protocol in use. The sender counts timeouts and the wall-clock transfer time, which are the performance metrics of this report.

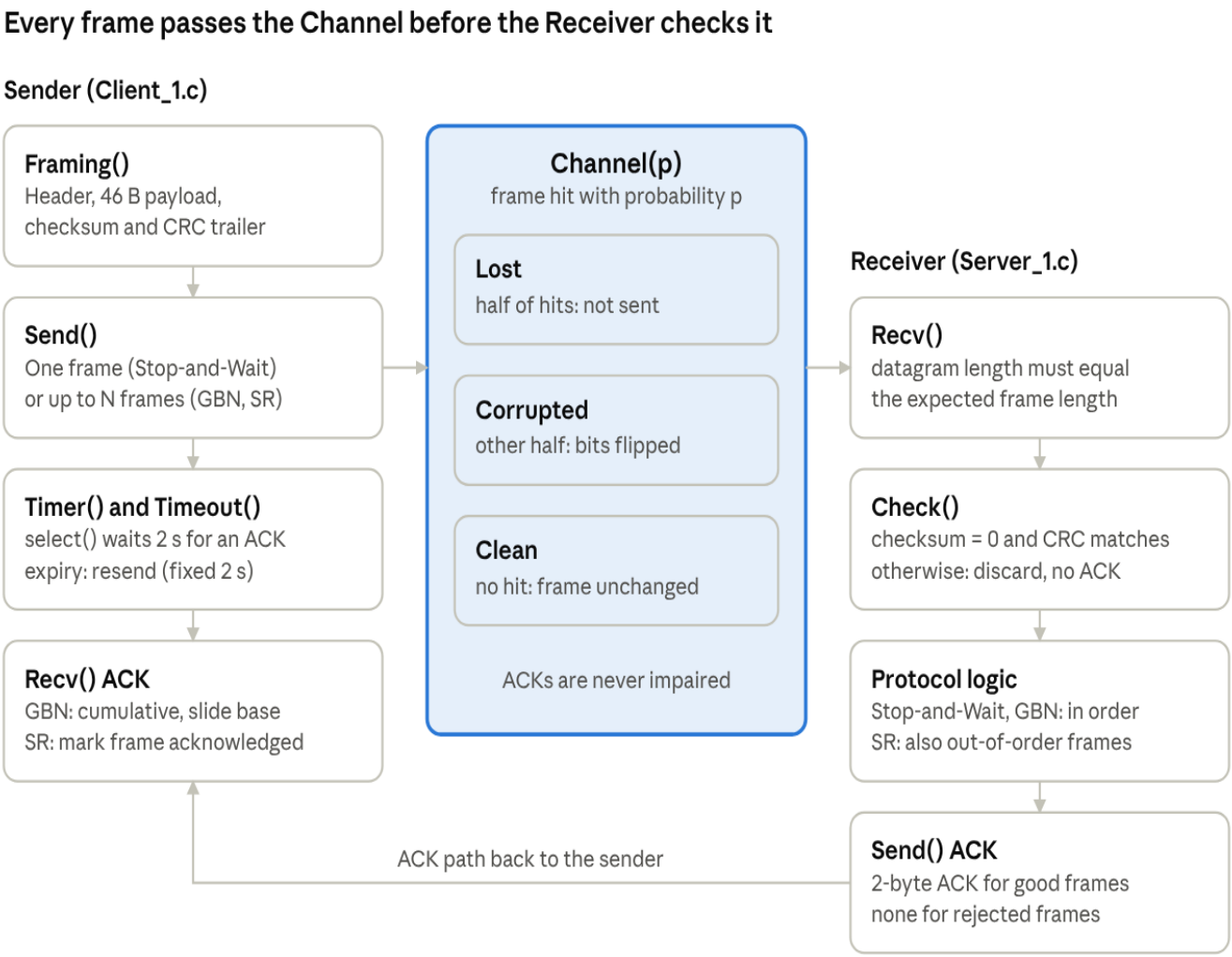
2.2 Frame and ACK format

Field	Size (bytes)	Notes
Source MAC	6	Present in the header, left zero-filled
Destination MAC	6	Present in the header, left zero-filled
Length	2	Number of valid payload bytes, network byte order
Sequence number	2	16-bit; Stop-and-Wait alternates 0 and 1
Payload	46	Next 46 bytes of the file; last block zero-padded
Checksum	2	16-bit checksum over header + payload
CRC	1 / 2 / 2 / 4	CRC-8 / CRC-10 / CRC-16 / CRC-32 over header + payload
Total	**65 / 66 / 66 / 68**	Receiver rejects any other length

The ACK frame is 2 bytes: a 16-bit acknowledgement number. In Stop-and-Wait it is the sequence number being acknowledged, in Go-Back-N it is cumulative (everything up to that number), and in Selective Repeat it acknowledges that single frame.

2.3 Structure diagram

The sender builds and impairs frames, the receiver checks them and answers with ACKs, and the three ARQ procedures differ only in the sender's window logic and



the receiver's reaction to out-of-order frames.

structure diagram · sender, channel and receiver

Figure 1. Procedural organisation of the Sender and Receiver programs.

2.4 Protocol rules

Property	Stop-and-Wait	Go-Back-N	Selective Repeat
Sender window	1	N	N
Receiver window	1	1	N
ACK type	One per frame	Cumulative	Independent, one per frame
Out-of-order frame at receiver	Duplicate discarded, old ACK repeated	Discarded, last in-order ACK repeated	Accepted and buffered, individually ACKed
Corrupted frame at receiver	Discarded, no ACK	Discarded, no ACK	Discarded, no ACK
Sender action on timeout	Resend the frame	Resend from the oldest unacknowledged frame	Resend only unacknowledged frames

2.5 Input and output format

- **Sender input:** `./client <Receiver_IP> <Test_File>`, then prompts for the CRC width (8, 10, 16, 32), the error mode (1 = random error type, 2 = selected type 1 to 5), the protocol (1 = Stop-and-Wait, 2 = Go-Back-N, 3 = Selective Repeat), the window size N (Go-Back-N and Selective Repeat only) and the network error/loss probability p.
- **Receiver input:** `./server`, then the CRC width and the protocol, which must match the sender.
- **Sender output:** one line per event: frame sent, frame LOST or CORRUPTED by the channel, ACK received, TIMEOUT with the retransmission point, and at the end the total number of timeouts.
- **Receiver output:** one line per frame (accepted, discarded as duplicate or out-of-order, or rejected because of an error) and one line per ACK sent.
- **Test data:** `Test_1.txt`, 30,720 bytes, which gives 668 frames (667 full frames and one zero-padded frame of 38 bytes).

3. Implementation

3.1 Source files

File	Role
Client_1.c	Sender main program, menus, `injkterror()` and `Channel()`
Server_1.c	Receiver: length check, checksum and CRC check, ACK generation for the three protocols
stop_wait.h	`send_stop_and_wait()`
go_back_n.h	`send_go_back_n()`
selective_repeat.h	`send_selective_repeat()`
checksum_1.h, Crc_1.h	Checksum and CRC modules reused from Assignment 1; the CRC value is now kept in variables as wide as the CRC, which removes the truncation bug of Assignment 1

3.2 Methods required by the assignment

Assignment method	Implementation
Framing()	First part of each `send_*` function: fills length and sequence number, copies the 46-byte payload, appends checksum and CRC
Channel()	`Channel(p)` returns 1 with probability p. On a hit, half the frames are dropped (LOST) and half are corrupted with `injkterror()` (random type 2 to 5, or the selected type) and still sent
Send()	`sendto()`. Stop-and-Wait sends one frame; Go-Back-N and Selective Repeat send while `next_seq_num < base + N`
Timer()	`select()` on the socket with a 2 s timeval; expiry

Assignment method	Implementation
	means timeout
Timeout()	The timeout branch: resend from `base` (Go-Back-N), resend unacknowledged frames (Selective Repeat), resend the frame (Stop-and-Wait). The 2 s value is fixed, not recomputed from the measured RTT
Recv() (sender)	`recvfrom()` of the 2-byte ACK; updates `base` or the `acked[]` table
Recv() (receiver)	`recvfrom()` in the `main` loop of Server_1.c; the datagram length must equal the expected frame length
Check()	Checksum over header + payload + checksum must be 0 and the recomputed CRC must equal the received CRC; a failure of either discards the frame
Send() (receiver)	`sendto()` of the ACK, built according to the selected protocol

3.3 Code snippets

Channel model (Client_1.c and the `send_*` headers):

```
int Channel(float error_prob) {
    float random_val = (float)rand() / (float)RAND_MAX;
    return (random_val < error_prob);      /* 1 = frame hit
by the channel */
}

if (Channel(error_prob)) {
    if (rand() % 2 == 0) { /* LOST: nothing is sent */ }
    else { /* copy frame, injkterror(copy, len, type),
sendto(copy) */ }
} else sendto(sockfd, frame, len, 0, ...);
```

Go-Back-N sender: cumulative ACK and window reset (go_back_n.h):

```
recvfrom(sockfd, &ack, sizeof(ack), 0, ...);
if (ack.ack_no >= base && ack.ack_no < total_frames)
    base = ack.ack_no + 1;                /* cumulative ACK
slides the window */
...
/* select() timed out */
next_seq_num = base;                      /* go back:
resend the whole window */
timeout_count++;
```

Selective Repeat sender: independent ACKs (selective_repeat.h):

```
acked[ack.ack_no] = 1;                    /* this frame
only */
while (base < total_frames && acked[base]) base++;
...
/* on timeout: next_seq_num = base; frames with acked[] set
are skipped */
```

Receiver: the two reactions to a good frame (Server_1.c):

```
/* Go-Back-N */
if (received_seq == expected_seq) { ack.ack_no =
expected_seq; expected_seq++; }
else ack.ack_no = expected_seq - 1;       /* out of order:
discard, repeat last ACK */

/* Selective Repeat */
ack.ack_no = received_seq;                /* ACK every good
frame */
if (received_seq == expected_seq) expected_seq++;
```

4. Test Cases

Unless stated, the tests use `Test_1.txt` (668 frames), CRC-16 and window N = 4.

#	Test	Setting / sample data	What is checked
T1	Error-free baseline	p = 0, all three protocols	File delivered, 0 timeouts, transfer time of each protocol

#	Test	Setting / sample data	What is checked
T2	Propagation to ACK time	p = 0, Stop-and-Wait	Time per frame (one round trip) and sender idle time
T3	Go-Back-N and Selective Repeat under errors	Random error mode, p = 0.1, 0.2, 0.3, 0.4, 0.5	Timeouts and transfer time of each protocol
T4	Stop-and-Wait under errors	Same p values	Timeouts, transmissions and estimated transfer time
T5	Corrupted frame	Error mode 2, error types 2 to 5	Receiver prints REJECTED, sends no ACK, the sender times out and retransmits
T6	Lost frame	Frame dropped by the channel	Timeout followed by retransmission (Stop-and-Wait one frame, Go-Back-N window, Selective Repeat unacknowledged frames)
T7	Out-of-order frame	Frame k lost, k+1 arrives	Go-Back-N discards it and repeats the last ACK; Selective Repeat ACKs it individually
T8	Duplicate frame	Retransmission after a lost ACK path or late ACK	Stop-and-Wait discards the duplicate and repeats the ACK
T9	Last block	File size not a multiple of 46	Final 38-byte block is zero-padded and the length field holds 38
T10	Mismatche	Sender and	Receiver reports a malformed frame

#	Test	Setting / sample data	What is checked
	d settings	receiver choose different CRC widths	(length mismatch)
T11	End of file	Last frame acknowledged	`DONE` datagram ends the receiver and the sender prints its metrics

T1 to T4 produced the measurements in Section 5; T5 to T11 check behaviour from the console logs.

5. Results

5.1 Setup and performance metrics

The metrics are the total transfer time at the sender, the number of timeouts, the time per frame, and the transmission efficiency (useful frames divided by all frames put on the channel, including lost, corrupted and retransmitted ones). Times for p = 0 and for Go-Back-N and Selective Repeat with p = 0.1 to 0.5 are the lab measurements of the sheet `ASSIGNMENT_2.xlsx`: one run per point, N = 4, CRC-16.

5.2 Error-free efficiency and ACK waiting (p = 0)

Protocol	Window N	Timeouts	Total transfer time	Time per frame	Speed-up over Stop-and-Wait
Stop-and-Wait	1	0	16.752 s	25.08 ms	1.00x
Go-Back-N	4	0	9.315 s	13.94 ms	1.80x
Selective Repeat	4	0	9.315 s	13.94 ms	1.80x

In Stop-and-Wait the time per frame (25.08 ms) is one full round trip from sending a frame to receiving its ACK, and the sender is idle for that whole time.

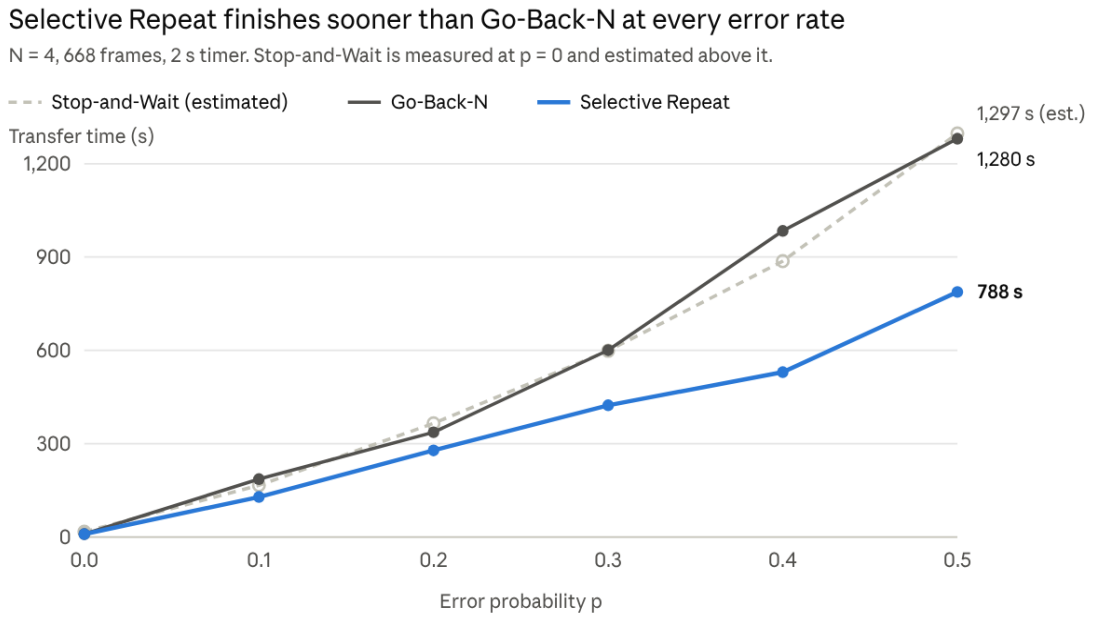
Go-Back-N and Selective Repeat send four frames before waiting, so the idle time is shared by four frames. Without errors the two behave identically because no frame is ever retransmitted and every ACK arrives in order.

Protocol	Sender behaviour	Receiver buffering	Sender idle time
Stop-and-Wait	Sends 1 frame, then halts until the ACK or the timer	None (expects 0 or 1)	At least one RTT per frame
Go-Back-N	Sends N frames, then waits for ACKs	None; out-of-order frames discarded	One RTT shared by N frames
Selective Repeat	Sends N frames, retransmits only unacknowledged ones	Out-of-order frames accepted	One RTT shared by N frames, and no wasted resends

5.3 Efficiency for error probabilities 0.1 to 0.5

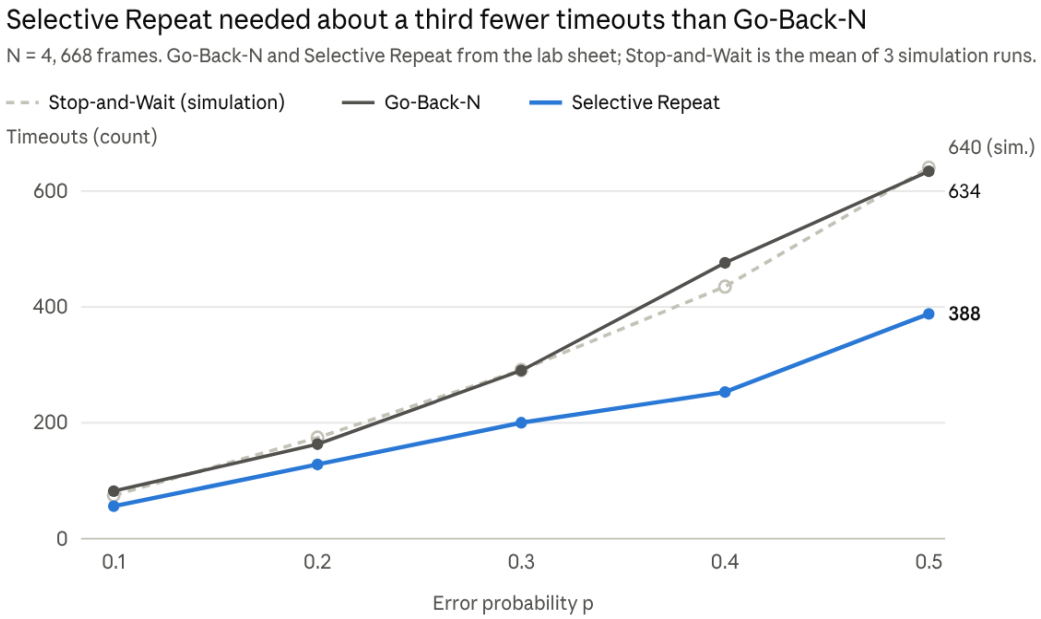
Error p	GBN timeouts	SR timeouts	Timeouts saved by SR	GBN time	SR time	Time saved by SR
0.1	82	56	31.7%	3m 6.169s (186.2 s)	2m 8.628s (128.6 s)	30.9%
0.2	163	128	21.5%	5m 36.690s (336.7 s)	4m 38.345s (278.3 s)	17.3%
0.3	290	200	31.0%	10m 0.591s (600.6 s)	7m 3.520s (423.5 s)	29.5%

Error p	GBN timeouts	SR timeouts	Timeouts saved by SR	GBN time	SR time	Time saved by SR
0.4	476	253	46.8%	16m 24.174s (984.2 s)	8m 50.072s (530.1 s)	46.1%
0.5	634	388	38.8%	21m 20.423s (1,280.4 s)	13m 7.737s (787.7 s)	38.5%



ASSIGNMENT_2.xlsx (Table 1, Table 2) and the supplementary Stop-and-Wait estimate of Section 5.4

Figure 2. Total transfer time against error probability.



ASSIGNMENT_2.xlsx (Table 2) and the supplementary simulation of Section 5.4

Figure 3. Number of timeouts against error probability.

Each timeout waits for the fixed 2 s timer, so the transfer time follows the timeout count closely. The prediction below uses T0 = 9.315 s (the error-free time) and Tout = 2 s:

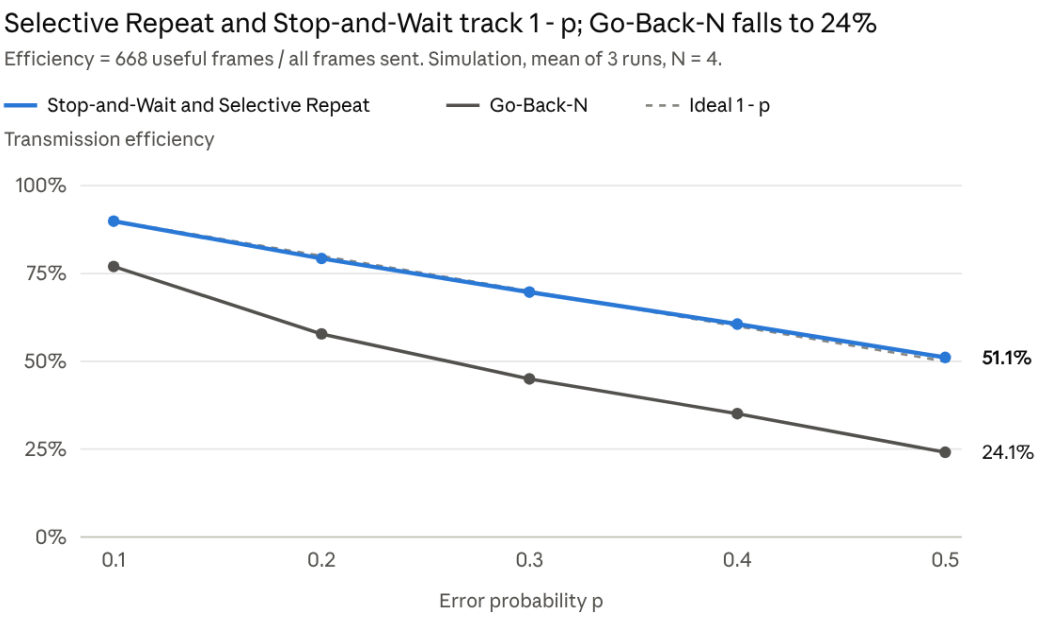
$$T(p) \approx T_0 + T_{out} . N_{timeouts}$$

Error p	GBN predicted (s)	GBN measured (s)	Difference	SR predicted (s)	SR measured (s)	Difference
0.1	173.3	186.2	-6.9%	121.3	128.6	-5.7%
0.2	335.3	336.7	-0.4%	265.3	278.3	-4.7%
0.3	589.3	600.6	-1.9%	409.3	423.5	-3.4%
0.4	961.3	984.2	-2.3%	515.3	530.1	-2.8%
0.5	1,277.3	1,280.4	-0.2%	785.3	787.7	-0.3%

5.4 Stop-and-Wait under errors and transmission efficiency (supplementary)

The sheet has no Stop-and-Wait run with errors. To complete the comparison, the sender and receiver sources were re-run unchanged except that the 2 s timer was shortened to 50 ms, with three random seeds per point. Timeouts and transmissions are counts, so they do not depend on the timer length as long as ACKs arrive in time; for Go-Back-N and Selective Repeat the timeout counts of this simulation are within 14% of the lab sheet. The Stop-and-Wait time is estimated as 16.752 s + 2 s x timeouts, the model checked above. **Replace these estimates with measured Stop-and-Wait runs if you have them.**

Error p	S&W timeouts (mean)	S&W estimated time	Frames sent: S&W	Frames sent: GBN	Frames sent: SR	Efficiency S&W and SR	Efficiency GBN
0.1	74.7	166.2 s (2m 46s)	743	867	743	89.9%	77.0%
0.2	174.3	365.4 s (6m 5s)	842	1,155	842	79.3%	57.8%
0.3	291.0	598.8 s (9m 59s)	959	1,486	959	69.7%	45.0%
0.4	435.0	886.8 s (14m 47s)	1,103	1,905	1,103	60.6%	35.1%
0.5	640.3	1,297.4 s (21m 37s)	1,308	2,775	1,308	51.1%	24.1%



supplementary simulation of Section 5.4 · 3 seeds per point

Figure 4. Transmission efficiency against error probability, with the ideal value 1 - p.

6. Analysis

Error-free case. Pipelining cut the time per frame from 25.08 ms to 13.94 ms, a speed-up of 1.80 for N = 4. The ideal speed-up is 4. The shortfall comes from the sender loop: it sends a window, then stays in its ACK loop until every frame of that window is acknowledged, and only then sends the next window. The window therefore does not slide while ACKs arrive, so part of each round trip is still idle.

Time is dominated by timeouts. At p = 0.5 the timeouts account for about 99% of the Go-Back-N time (2 s x 634 = 1,268 s of 1,280 s) and 98% of the Selective Repeat time. The prediction T0 + 2 s x timeouts is within 7% of every measured point. A timer of about twice the measured round trip (50 ms) instead of 2 s would bring the Go-Back-N run at p = 0.5 down from about 21 minutes to roughly 41 s (9.3 s + 634 x 0.05 s), an estimate rather than a measurement.

Go-Back-N against Selective Repeat. Selective Repeat needed 22% to 47% fewer timeouts and finished 17% to 46% sooner at every error probability. A lost

frame in Go-Back-N forces every later frame of the window to be resent, and those resent frames can be lost again. In the supplementary run Go-Back-N sent 2,775 frames to deliver 668 at p = 0.5 (efficiency 24%), while Selective Repeat sent 1,308 (51%), which is the ideal 1 - p. Stop-and-Wait reaches the same 1 - p efficiency but at a far lower speed, since every frame costs a full round trip.

Variation between runs. Each lab point is one run, so the gap is not smooth: the time saved by Selective Repeat drops from 30.9% at p = 0.1 to 17.3% at p = 0.2 before rising again. In the three-seed simulation the timeout count at one probability varied by about 10 to 15% between seeds. Repeating each point several times and reporting the mean would remove this.

Known bugs and limitations.

1. The window does not slide: new frames are sent only after the whole window is acknowledged (see the error-free case).
2. `Timeout()` is not implemented as the assignment describes: the 2 s timer is fixed, not recomputed from the measured RTT, and one timer is re-armed on every ACK instead of one timer per frame.
3. The channel only loses or corrupts frames. It adds no random delay, and ACKs are never lost or corrupted, so errors on the ACK path are not tested.
4. The Selective Repeat receiver only counts and labels frames as buffered. It has no reorder buffer and does not check that a frame lies inside its window.
5. MAC addresses are zero-filled, the sequence number is 16 bits instead of the 1 byte of the assignment figure, and the frame tables hold at most 1,000 frames.
6. In Go-Back-N, `expected_seq - 1` becomes 65535 when the first frame is out of order; the sender ignores it, so it is harmless.
7. The receiver waits for the `DONE` datagram; if UDP drops it the receiver never ends.

Possible improvements. Restructure the sender loop so it sends whenever `next_seq_num < base + N` while polling for ACKs; keep a send time per frame and compute the timeout from a smoothed RTT estimate; add random delay and ACK loss to `Channel()`; add a real reorder buffer and window check to the

Selective Repeat receiver; optionally add NAKs for faster recovery; and repeat each measurement several times.

7. Comments

This assignment showed how the three ARQ schemes trade sender complexity, receiver memory and bandwidth. The measurements made clear that with a long fixed timer the number of timeouts matters more than anything else, and that Selective Repeat wastes the least bandwidth. Getting the window and timer logic right in a single-threaded loop with `select()` was the hardest part; the framing and error detection came directly from Assignment 1. The assignment was of moderate difficulty. A useful improvement would be to give a target RTT and ask for an adaptive timeout, since the timer value decides the results so strongly.

Appendix: source files

`Client_1.c`, `Server_1.c`, `stop_wait.h`, `go_back_n.h`, `selective_repeat.h`, `checksum_1.h`, `Crc_1.h`, `Test_1.txt` (30,720 bytes), `ASSIGNMENT_2.xlsx` (measurements).